

```

# --- STEP 1: LIBRARIES ---
!pip install requests pandas folium matplotlib seaborn geopandas shapely

import requests
import pandas as pd
import folium
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
import shapely.wkb

# Style set up
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = [12, 6]

print("Starting Full Temporal & Spatial Analysis (500m / 6 Months)...")

# --- STEP 2: DATA FETCHING ---
days_back = 180
past_date = (datetime.now() - timedelta(days=days_back)).strftime('%Y-%m-%d')
ch_lng, ch_lat = 39.9526, -75.1652
radius_in_degrees = 0.0045 # ~500 meters

sql_query = f"""
SELECT the_geom, text_general_code, dispatch_date_time
FROM incidents_part1_part2
WHERE dispatch_date_time >= '{past_date}'
AND text_general_code IN ('Theft from Vehicle', 'Other Theft', 'Burglary Residential', 'Burglary Commercial')
AND ST_Within(the_geom, ST_SetSRID(ST_MakePoint({ch_lng}, {ch_lat}), 4326), {radius_in_degrees})
ORDER BY dispatch_date_time ASC
"""

api_url = f"https://ph1.carto.com/api/v2/sql?q={sql_query}"

try:
    response = requests.get(api_url)
    data = response.json()
    if 'error' in data:
        sql_query = sql_query.replace("incidents_part1_part2", "inc_part1_part2")
        api_url = f"https://ph1.carto.com/api/v2/sql?q={sql_query}"
        response = requests.get(api_url).json()
        df = pd.DataFrame(response['rows'])
    else:
        df = pd.DataFrame(data['rows'])
except:
    df = pd.DataFrame()

# --- STEP 3: DATA PROCESSING & TIME ANALYSIS ---
if not df.empty:
    # Geometry fix
    df['geometry'] = df['the_geom'].apply(lambda x: shapely.wkb.loads(x, hex=True) if pd.notnull(x) else None)
    df['lat'] = df['geometry'].apply(lambda geom: geom.y if geom else None)
    df['lng'] = df['geometry'].apply(lambda geom: geom.x if geom else None)

    # Time Features Extraction
    df['dispatch_date_time'] = pd.to_datetime(df['dispatch_date_time'])
    df['hour'] = df['dispatch_date_time'].dt.hour
    df['day_name'] = df['dispatch_date_time'].dt.day_name()
    df['date_only'] = df['dispatch_date_time'].dt.date

    # Sorting days for the graph
    day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
    df['day_name'] = pd.Categorical(df['day_name'], categories=day_order, ordered=True)

# --- STEP 4: VISUALIZATIONS ---

# A. INTERACTIVE MAP (500m)
print("\n1. Generating Map...")
m = folium.Map(location=[ch_lat, ch_lng], zoom_start=16, tiles='CartoDB positron')
folium.Circle(location=[ch_lat, ch_lng], radius=500, color='blue', fill=True, fill_opacity=0.05).add_to(m)
for _, row in df.iterrows():
    folium.CircleMarker(location=[row['lat'], row['lng']], radius=5, color='red', fill=True,
        popup=f"{row['text_general_code']} at {row['hour']}:00").add_to(m)

display(m)

```

```

# B. CRIME BY HOUR (When do they strike?)
print("\n2. Analyzing Peak Hours...")
plt.figure(figsize=(12, 5))
sns.countplot(data=df, x='hour', palette='Reds_r')
plt.title('Crime Frequency by Hour of Day (24-Hour Cycle)', fontsize=15, fontweight='bold')
plt.xlabel('Hour (0 = Midnight, 12 = Noon, 23 = 11 PM)')
plt.show()

# C. HEATMAP (Day vs Hour) - The Pattern
print("\n3. Generating Heatmap (Day vs Hour)...")
pivot_table = df.pivot_table(index='day_name', columns='hour', values='text_general_code', aggfunc='count').fillna(0)
plt.figure(figsize=(15, 6))
sns.heatmap(pivot_table, annot=True, cmap='YlOrRd', fmt='g')
plt.title('Crime Intensity Heatmap: Day of Week vs. Time', fontsize=15, fontweight='bold')
plt.show()

# --- FINAL STATS ---
peak_hour = df['hour'].mode()[0]
peak_day = df['day_name'].mode()[0]

print(f"\n--- FINAL INTELLIGENCE REPORT ---")
print(f"Total Incidents (Last 6 Months): {len(df)}")
print(f"Most Dangerous Day: {peak_day}")
print(f"Peak Crime Time: {peak_hour}:00 Hours")
print(f"Primary Pattern: {df['text_general_code'].value_counts().idxmax()}")

else:
    print("No data found for the selected 500m zone.")

```

